1. Indexing

What: Access a **single element** using position.

```
arr[2]
arr[1, 3]
```

Mental model:

> "Exact address chahiye."

Use when you know **exact location**.

---

2. Slicing

What: Access a **continuous range** of elements.

```
arr[2:6]
arr[:, 1:3]
```

Mental model:

> "Ek continuous piece kaatna."

Rules:

- `start:stop:step`
- Stop is **exclusive**

Fast. Clean. Most used.

---

3. Fancy Indexing

What: Select elements using a **list/array of indices**.

```
arr[[0, 3, 5]]
```

Mental model:

> "Mujhe yeh-yeh positions chahiye, order meri marzi."

Not continuous. Custom selection.

Returns a **copy**, not a view.

---

4. Boolean Masking

What: Select elements based on a **condition**.

```
arr[arr > 30]
```

Mental model:

> "Condition pass kare wahi lo."

Extremely powerful:

- Filtering
- Cleaning data
- Rule-based selection

Core data-science skill.

---

5. Array Shape

What: Defines **structure**, not data.

```
arr.shape
```

Example:

```
(3, 4)
```

Mental model:

> "Data ka blueprint."

Wrong shape = wrong meaning.

---

6. `ravel()` vs `flatten()`

`ravel()`

```
arr.ravel()
```

- Returns **view** if possible
- Faster
- Changes may reflect back

Mental model:

> "Same data, different lens."

`flatten()`

```
arr.flatten()
```

- Always returns **copy**
- Safer
- Slightly slower

Mental model:

> "Nayi independent copy."

One-glance comparison table

Concept	Purpose	Key idea
Indexing	Single value	Exact address
Slicing	Continuous block	Range cut
Fancy indexing	Custom positions	Pick & choose
Boolean masking	Condition-based	Filter
Shape	Structure	Meaning of data
ravel	Fast flatten	View
flatten	Safe flatten	Copy

Core rule (remember this)

Selection decides correctness. Shape decides meaning. Copy vs view decides bugs.

If you understand that, you're using NumPy like an engineer, not a beginner.